

Abschlussbericht: TerminPilot – ein plattformübergreifender Terminplaner

Projektzeitraum: 15.04.2026 – 24.09.2026 (verlängert laut CR-02, ursprünglich bis 30.07.2026 geplant)

Team: Lotta Scholze (Product Owner), Leon Reusch (Tech Lead), Jannis Dickel (Fullstack), Leon Huttarsch (QA), Marlene Glaab (QA)

Dozent: Hr. Berner

Abgabedatum: 24.09.2026

1. Summary

TerminPilot wurde als Webapplikation zur plattformübergreifenden Terminfindung und -erstellung erfolgreich fertiggestellt und in Betrieb genommen. Alle im Projektantrag definierten Abnahmekriterien wurden erfüllt, alle KPI-Zielwerte erreicht oder übertroffen. Der ursprüngliche Zeitplan konnte aufgrund technischer Herausforderungen bei der Kalenderintegration nicht gehalten werden; das Projektteam reagierte mit drei dokumentierten Änderungsanträgen (CR-01 bis CR-03), wodurch Projektsteuerung, Zeitplan und Arbeitsweise erfolgreich an die tatsächlichen Rahmenbedingungen angepasst wurden. Das Projekt schließt mit dem vorliegenden Bericht fristgerecht zum neuen Endtermin 24.09.2026 ab.

2. Projektüberblick: Soll vs. Ist

2.1 Ursprüngliche Planung (Projektantrag, 15.04.2026)

Der Projektantrag sah eine Laufzeit vom 15.04. bis 30.07.2026 vor, gegliedert in Vorbereitungs-/Planungsphase, Konzeptionsphase mit Design-Sprint, Entwicklungsphase (Mai/Juni) und Projektabschluss (Juli). Steuerung sollte über Jira/Confluence in einem an Scrum angelehnten Modell erfolgen.

2.2 Tatsächlicher Verlauf inklusive Änderungsanträge

Change Request	Datum	Änderung	Grund
CR-01	20.04.2026	Jira/Confluence → GitHub (Issues, Projects, Milestones)	Wegfall kostenfreier Nutzung von Jira/Confluence

CR-02	27.05.2026	Projektlaufzeit verlängert bis 24.09.2026	Verzögerung bei der Kalenderintegration, Restaufwand höher als geschätzt, Zeitliche Möglichkeit gegeben
CR-03	21.07.2026	Scrum-ähnlich → Kanban mit bedarfsorientierten Meetings	Unregelmäßige Verfügbarkeit in der Theoriephase

Alle drei Änderungsanträge lassen sich auf konkrete Erkenntnisse aus den laufenden Weeklys bzw. den monatlichen Review-/Retro-Terminen zurückführen (siehe Abschnitt 7).

2.3 Meilenstein-Vergleich (Original vs. Tatsächlich)

Meilenstein	Ursprünglich geplant	Tatsächlich erreicht	Abweichung
M1: Anforderungen	22.04.2026	~20.04.2026	keine
M2: Design-Sprint	05.05.2026	~27.04.2026	keine (früher)
M3: Funkt. Zwischenstand	25.05.2026	~29.06.2026	ca. 5 Wochen
M4: Optionale Funktionen	22.06.2026	~17.08.2026	ca. 8 Wochen
M5: QS abgeschlossen	13.07.2026	~07.09.2026	ca. 8 Wochen
M6: Inbetriebnahme	20.07.2026	~14.09.2026	ca. 8 Wochen
M7: Abschlussbericht	27.07.2026	24.09.2026	ca. 8 Wochen

Hauptursache der Verzögerung: Die Anbindung des Apple-Kalenders erwies sich als deutlich komplexer als in der Aufwandsschätzung angenommen; zusätzlich verschärfte sich die Verzögerung durch abnehmende, unregelmäßige Verfügbarkeit des Teams in der Theoriephase – generell wurde durch CR-02 aber auch mit mehr Zeit geplant.

3. Zielerreichung

3.1 SMART-Ziele

Ziel	Erreicht?	Anmerkung
Funktionale Anwendung zur Terminfindung	Ja	Alle Kernfunktionen (Vergleichen, Anfragen, Erstellen) live
Integration externer Kalenderdienste	Ja	Google und Apple angebunden, Sync-Erfolgsquote über Zielwert

Sicherstellung der Benutzerfreundlichkeit	Ja	Usability-Bewertung über Zielwert
---	----	-----------------------------------

3.2 KPI-Ergebnisse

KPI	Zielwert	Ist-Wert
Nutzerfreundlichkeit (positive Bewertung)	≥80 %	90 %
Erfolgsquote Kalendersynchronisation	≥95 %	100 %
Fehlerrate funktionale Tests	≤5 %	0 %
Systemverfügbarkeit	≥99 %	100 %
Response-Time	≤2 Sek.	1.66 Sek.

3.3 Abnahmekriterien

Alle im Projektantrag definierten Abnahmekriterien sind erfüllt: zentrales Vergleichen von Kalendern verschiedener Systeme, unterstützte Terminfindung, Erstellen von Terminanfragen, zentrale Terminerstellung, korrekte Verarbeitung der Kalenderdaten, keine funktionseinschränkende oder sicherheitsrelevanten Fehler, Einhaltung der KPIs.

4. Abweichungsanalyse

Die zeitliche Abweichung von insgesamt rund acht Wochen gegenüber dem ursprünglichen Endtermin lässt sich auf zwei Hauptfaktoren zurückführen:

1. **Technische Komplexität der Kalender-Integration:** Insbesondere die Anbindung des Apple-Kalenders benötigte deutlich mehr Aufwand als in der PERT-Schätzung des Projektantrags vorgesehen. Das Team reagierte mit einem reduzierten Funktionsumfang, um den Kernnutzen frühzeitig sicherzustellen.
2. **Veränderte Verfügbarkeit in der Theoriephase:** Mit Beginn der Vorlesungszeit sank die planbare, gleichmäßige Verfügbarkeit des Teams gegenüber der Praxisphase spürbar. Dies wurde durch CR-03 (Umstellung auf Kanban) sowie durch bewusste Kapazitätsplanung rund um Vorlesungszeiten und die Klausurenphase ab Mitte September aufgefangen.

Beide Ursachen wurden zunächst in den zweiwöchigen Weeklys sichtbar und anschließend im jeweils folgenden monatlichen Review-Termin offen besprochen und über einen dokumentierten Änderungsantrag (CR-02 bzw. CR-03) strukturiert bearbeitet, statt informell "mitzulaufen".

5. Ressourcen- und Aufwandsauswertung

Die ursprüngliche PERT-Schätzung im Projektantrag ging von einem Gesamtaufwand von rund 88 Personentagen aus (74 PT zzgl. 20 % Risikopuffer). Der tatsächliche Aufwand lag insbesondere in den Arbeitspaketen "iOS-Kalender-API" und "Synchronisationslogik" spürbar über der ursprünglichen

Schätzung; der eingeplante Risikopuffer wurde vollständig aufgebraucht, was die Notwendigkeit von CR-02 zusätzlich untermauerte.

Kapazitätsseitig zeigte sich, dass die im Projektantrag angenommene, gleichmäßige wöchentliche Verfügbarkeit (1–2 Personentage je Person) in der Praxisphase realistisch war, in der anschließenden Theoriephase durch Vorlesungen und Klausuren jedoch schwankte.

6. Technische Umsetzung (zusammengefasst)

Die Anwendung wurde als Webapplikation mit einem Kotlin/Spring-Boot-Backend und einer React-basierten Benutzeroberfläche umgesetzt. Architektonische Entscheidungen wurden mittels ADRs dokumentiert. Für die Kalenderanbindung kommen die offiziellen Schnittstellen von Google sowie ein CalDAV-basierter Ansatz für Apple-Kalender zum Einsatz. Die gesamte Entwicklung wird über eine automatisierte Qualitätssicherung abgesichert: Vor jeder Änderung prüfen automatische Werkzeuge Codeformat, Testabdeckung und mögliche Sicherheitslücken; jedes veröffentlichte Release wird automatisiert gebaut, versioniert und bereitgestellt.

7. Risikomanagement – Rückblick

Risiko aus dem Risikoregister	Eingetreten?	Umgang
API-Integration schlägt fehl	Teilweise (deutliche Verzögerung, kein Totalausfall)	Erstmal vereinfacht implementiert
Aufwand unterschätzt	Ja (insb. iOS-Integration)	Risikopuffer genutzt, CR-02 als strukturierte Reaktion
Synchronisationsfehler	Nein	Keine Probleme
Geringe Nutzerakzeptanz	Nein	Usability-Ziel deutlich übertroffen
Datenschutzproblem	Nein	Datenpannenprotokoll vorsorglich erstellt
Team-Ausfall	Nein (nur einzelne, kurzfristige Abwesenheiten)	Vertretungsregelungen in den monatlichen Terminen

Der Notfall- und Eskalationsplan sowie das Datenpannenprotokoll kamen im Projektverlauf nicht zur Anwendung

8. Laufende Projektsteuerung

8.1 Zweiwöchentliche (Bi-)Weeklys

Die zweiwöchigen Weeklys dienen der reinen Fortschrittsabstimmung, dokumentiert als kurze Statuslogs (Erledigt / In Arbeit / Blocker / Nächste Schritte) in GitHub. Sie ermöglichen eine frühzeitige, kleinteilige Sichtbarkeit von Problemen, bevor diese im nächsten Monatsreview offiziell behandelt wurden.

Weekly	Datum	Zentrales Thema
1	29.04.	Anforderungen abgeschlossen, Design-Sprint-Vorbereitung
2	13.05.	Design-Sprint fertig, GitHub-Umstellung abgeschlossen
3	27.05.	Erste Anzeichen Apple-Integrationsproblem
4	10.06.	Apple-Blocker hält an, Neuschätzung angekündigt
5	24.06.	M3 erreicht, Mehr Zeit in Aussicht: CR-02 vorbereitet
6	08.07.	Apple-Anbindung fertig, CR-03 vorbereitet

8.2 Monatliche Review-/Retro-Termine

Über den Projektverlauf hinweg fand einmal im Monat ein kombinierter Termin statt, immer zuerst als Review (Status, Fakten), danach als Retro (Zusammenarbeit, Verbesserung). Aus diesen Terminen gingen Änderungsanträge sowie die wichtigsten technischen und organisatorischen Anpassungen hervor. Eine Zusammenfassung:

Termin	Kernergebnis
18.05.2026	Anforderungen und Design-Sprint erfolgreich, Grundstein für Zusammenarbeit gelegt
15.06.2026	Verzögerung bei M3 erkannt, Grundlage für CR-02
18.07.2026	Verlängerung wirkt, neue technische Herausforderungen (Wartezeiten), Grundlage für CR-03
15.08.2026	Erste Version veröffentlicht, Vorbereitung auf Klausurenphase
12.09.2026	QS abgeschlossen, Synchronisationsproblem sauber bewältigt
24.09.2026	Projektabschluss, alle Ziele erreicht

9. Stakeholder-Feedback

Endnutzer bewerteten insbesondere die anonymisierte Verfügbarkeitsanzeige und die automatische Übertragung akzeptierter Termine in den eigenen Kalender positiv. Rückmeldungen zu Kalender-Betreibern und Datenschutzerfordernungen wurden ohne größere Konflikte umgesetzt; die im Projektantrag identifizierten Konfliktpotenziale (z. B. Vertrauen in den Umgang mit Kalenderdaten) traten in der Praxis nicht in kritischem Ausmaß auf.

10. Lessons Learned

- **Zeitpuffer bei Fremdsystem-Integrationen:**
Externe Kalender-APIs sollten von Anfang an mit deutlich mehr Zeitpuffer geplant werden als ursprünglich naheliegend erscheint.
- **Feste Reflexionstermine trotz wechselnder Arbeitsweise:**
Der monatliche Review-dann-Retro-Rhythmus hat sich unabhängig vom zugrunde liegenden Steuerungsmodell (Scrum-ähnlich oder Kanban) bewährt und sollte in vergleichbaren Projekten von Beginn an fest eingeplant werden.
- **Wechselnde Verfügbarkeit einkalkulieren:**
Praxis- und Theoriephasen eines dualen Studiums unterscheiden sich stark in der planbaren Kapazität; dies sollte bereits in der ursprünglichen Aufwandsschätzung stärker berücksichtigt werden.
- **Automatisierte Qualitätssicherung ergänzt, ersetzt aber keine fachliche Prüfung:**
Automatische Prüfmechanismen haben viele Probleme früh abgefangen, konnten aber eine fachliche Prüfung nicht ersetzen – gezielte Prüfungen kritischer Komponenten bleiben notwendig.
- **Abwesenheiten einplanen statt improvisieren:**
Ein Termin sollte nie von einer einzelnen Person abhängen; das Vorgehen, fehlende Beiträge schriftlich vor- oder nachzureichen, hat sich bewährt.

11. Ausblick / Empfehlungen für Folgeprojekte

- Frühzeitige technische Machbarkeitsprüfung bei kritischen Drittanbieter-Schnittstellen, idealerweise vor der finalen Aufwandsschätzung.
- Grobe Berücksichtigung von Urlaubs- und Klausurzeiten bereits in der Projektplanungsphase.
- Übernahme des monatlichen Review-x-Retro-Formats als Standardvorgehen unabhängig vom gewählten Steuerungsmodell.
- Ergänzung der automatisierten Qualitätssicherung um gezielte, wiederkehrende Prüfungen besonders kritischer, aber selten geänderter Programmteile.